

Neural networks con simplicidad

Prof. M.A.Valle
E-MAIL: mauricio.valle.b@uai.cl

Por qué NN?

Supongamos que trabajamos en un banco en el depto de inversiones. Nos interesa saber la capacidad de inversión de nuestra base de clientes.

Por qué NN?

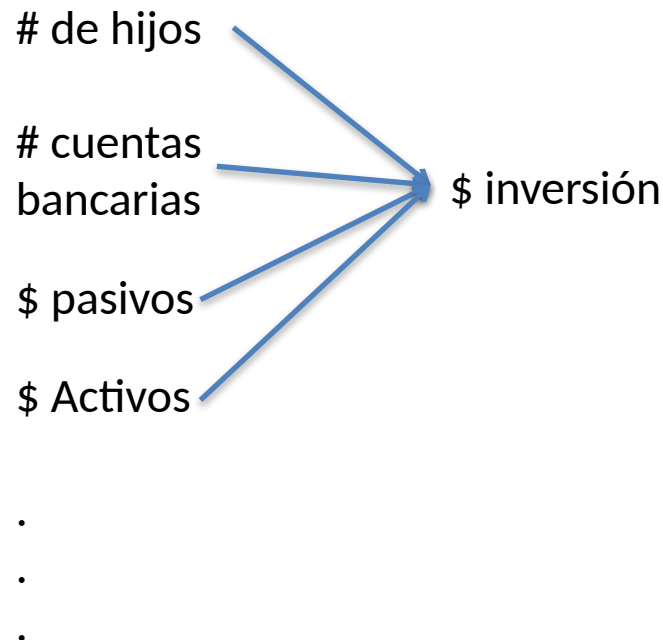
Supongamos que trabajamos en un banco en el depto de inversiones. Nos interesa saber la capacidad de inversión de nuestra base de clientes.

Podríamos formular, en base a información histórica, un modelo de regresión que nos permita entender cuál es la relación entre la inversión de nuestros clientes y sus variables (atributos):

Por qué NN?

Supongamos que trabajamos en un banco en el depto de inversiones. Nos interesa saber la capacidad de inversión de nuestra base de clientes.

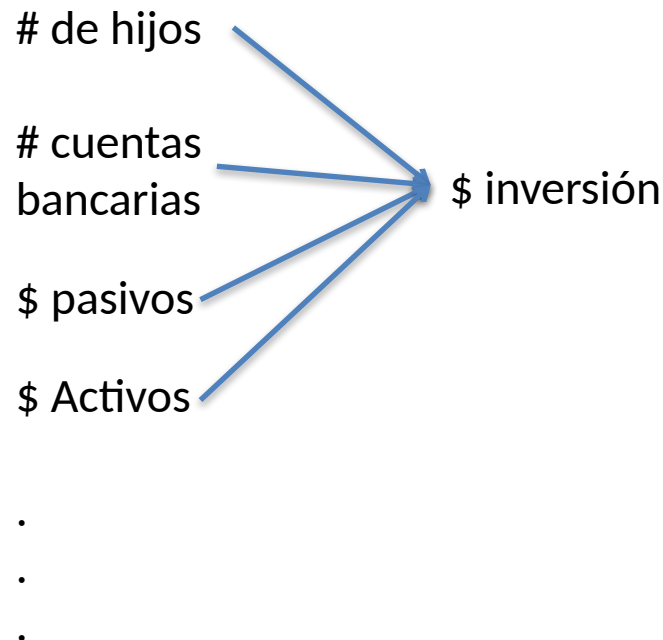
Podríamos formular, en base a información histórica, un modelo de regresión que nos permita entender cuál es la relación entre la inversión de nuestros clientes y sus variables (atributos):



Por qué NN?

Supongamos que trabajamos en un banco en el depto de inversiones. Nos interesa saber la capacidad de inversión de nuestra base de clientes.

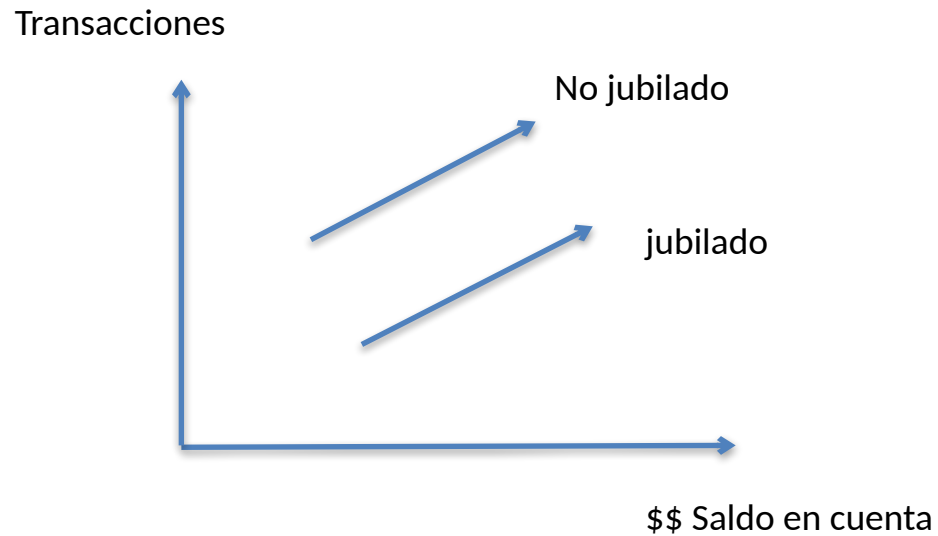
Podríamos formular, en base a información histórica, un modelo de regresión que nos permita entender cuál es la relación entre la inversión de nuestros clientes y sus variables (atributos):



¿qué
deficiencia
tendría
Este
modelo?

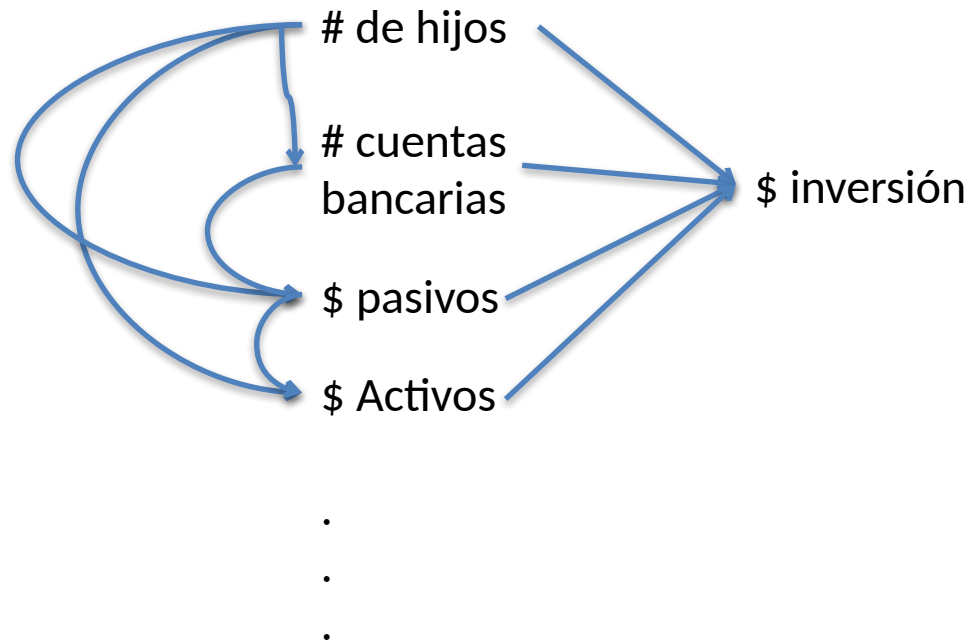
Por qué NN?

Esto es lo que vería un modelo lineal:
Aquí no hay posibilidad de interacciones.



Por qué NN?

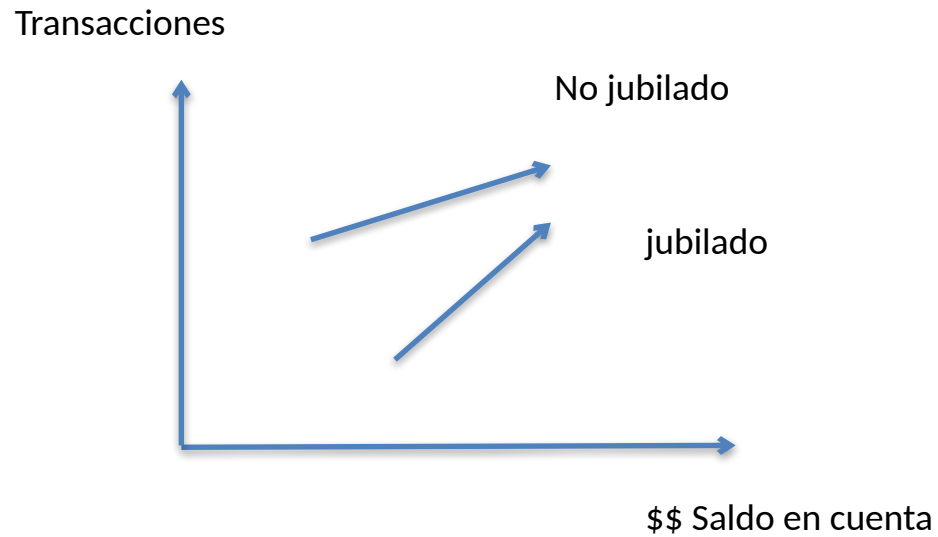
Con una NN, podemos tomar en cuenta MULTITUD de interacciones existentes entre variables, lo que da lugar la capacidad de identificar relaciones NO-LINEALES.



PERO....
¿qué
desventaja
tiene esta
aproximación?

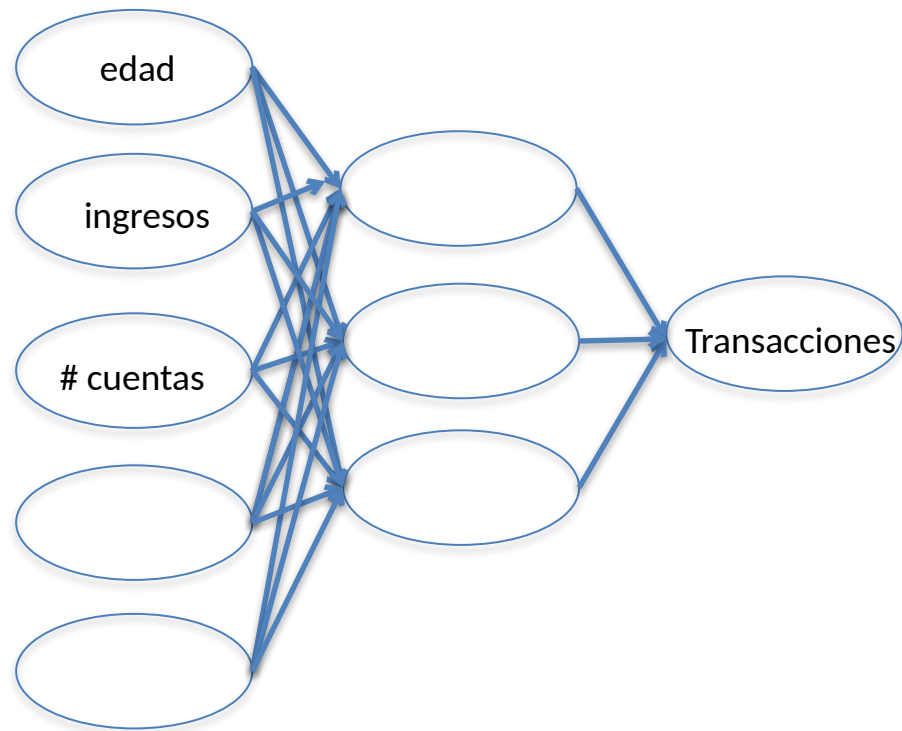
Por qué NN?

Un modelo con interacciones:



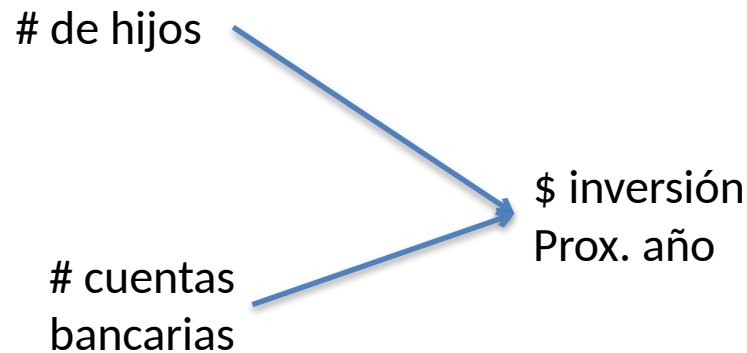
Por qué NN?

Con la NN, podemos fabricar un modelo que logre aprender de todas las intrincadas interacciones existentes entre las variables predictoras del número de transacciones, que dan lugar a patrones no lineales.



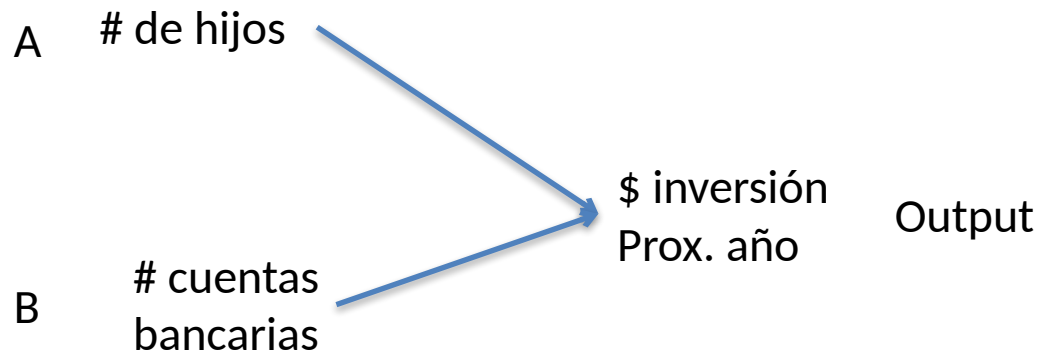
Ejemplo (la idea del forward propagation)

Supongamos que el monto de inversión depende del # de hijos y # num de cuentas bancarias.



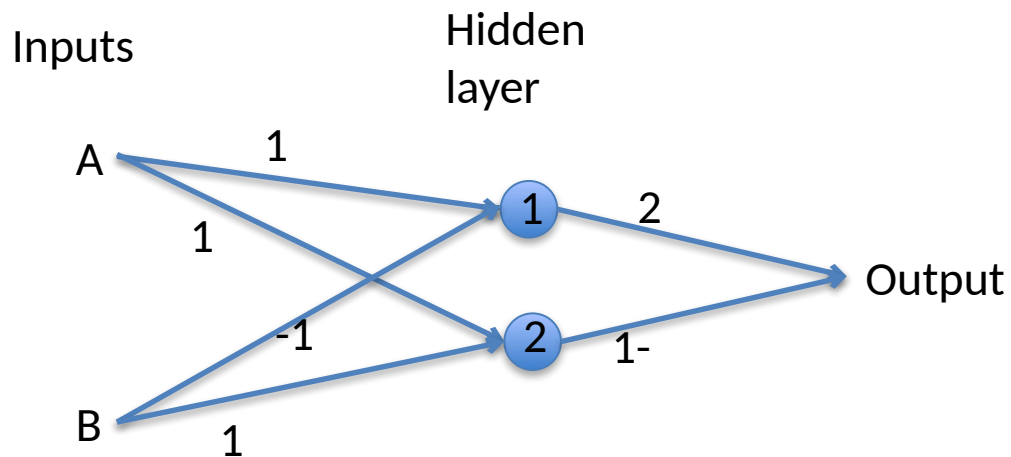
Ejemplo (la idea del forward propagation)

Supongamos que el monto de inversión depende del # de hijos y # num de cuentas bancarias.



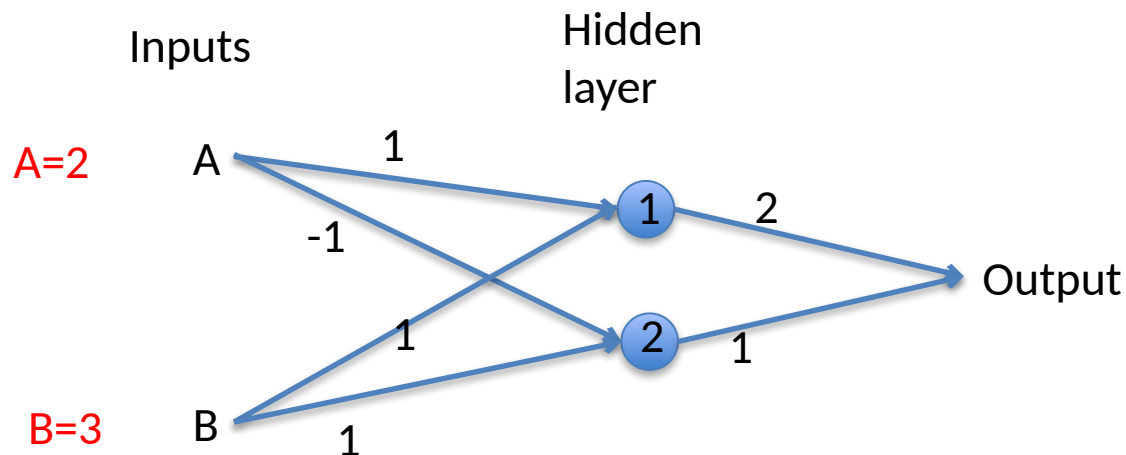
Ejemplo (la idea del forward propagation)

Veamos cómo sería un modelo de red neuronal para entender cómo se maneja la interacción entre variables.



Ejemplo (la idea del forward propagation)

Supongamos una INSTANCIA con vector (2,3), es decir, A=2, B=3.
En ese caso podemos calcular los valores en cada nodo:



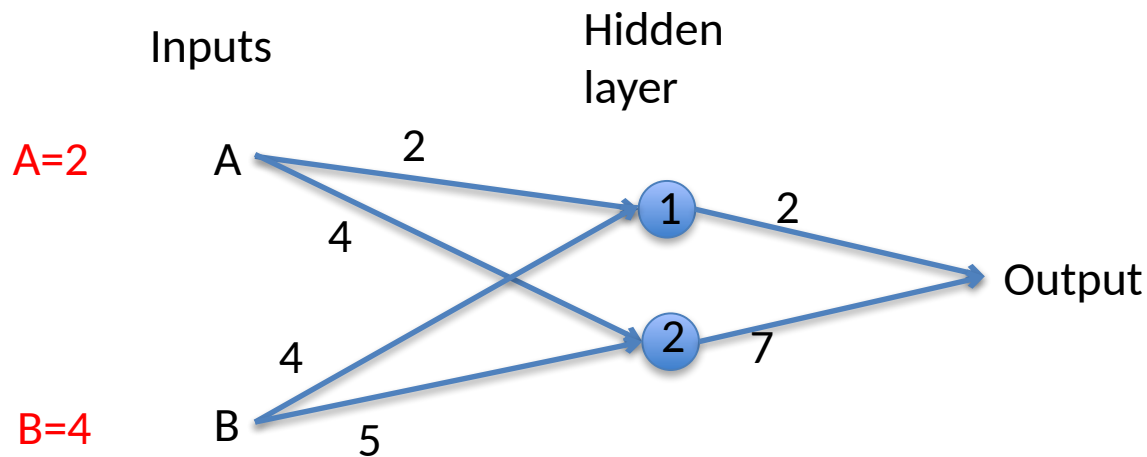
$$\begin{aligned}\text{node 1 value} &= \begin{bmatrix} 2 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 5 \\ \text{node 2 value} &= \begin{bmatrix} 2 & 3 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = 1\end{aligned}$$

$$\text{output} = \begin{bmatrix} 5 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = 11$$

Ver `1_forward_prop_example.ipynb`

Ejemplo (la idea del forward propagation)

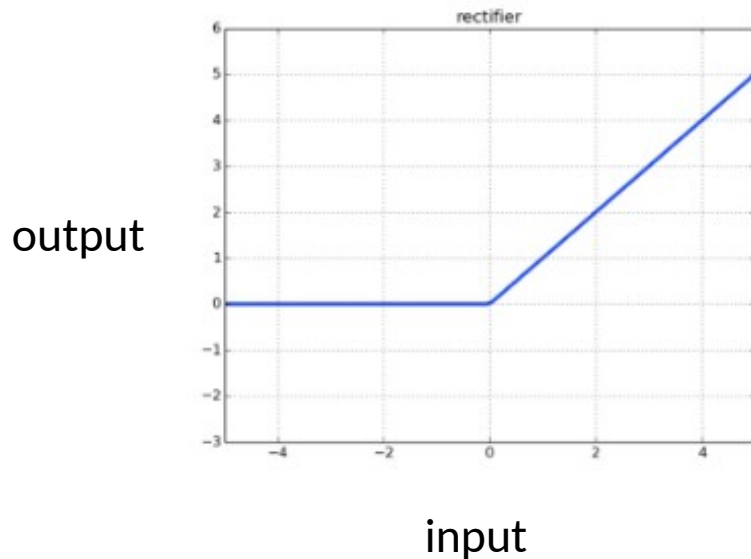
Hagamos otro ejemplo, a mano y luego utilizando Python modificando:
Ver 1_forward_prop_example.ipynb



Activation Functions

Para obtener el máximo beneficio de las NN, debemos incluir en la hidden layer, unas “funciones de activación”.

- Esta es la forma por la cual las NN son capaces de capturar las NO-LINEALIDADES.
- Por ejemplo, RELU function:

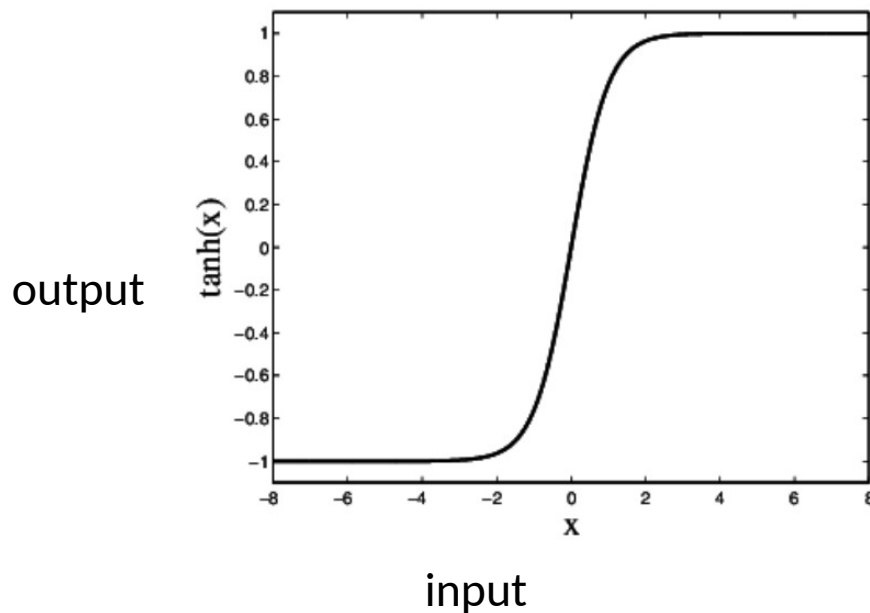


$$RELU(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

Activation Functions

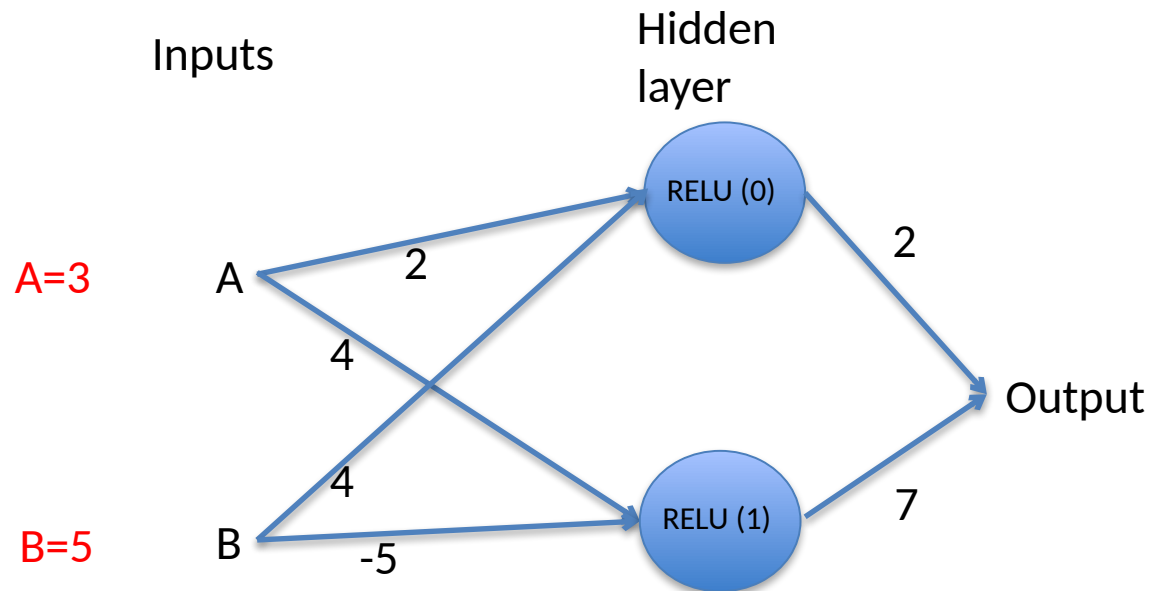
Para obtener el máximo beneficio de las NN, debemos incluir en la hidden layer, unas “funciones de activación”.

- Esta es la forma por la cual las NN son capaces de capturar las NO-LINEALIDADES.
- Por ejemplo, tangente hiperbólica ($\tanh$):



Activation Functions

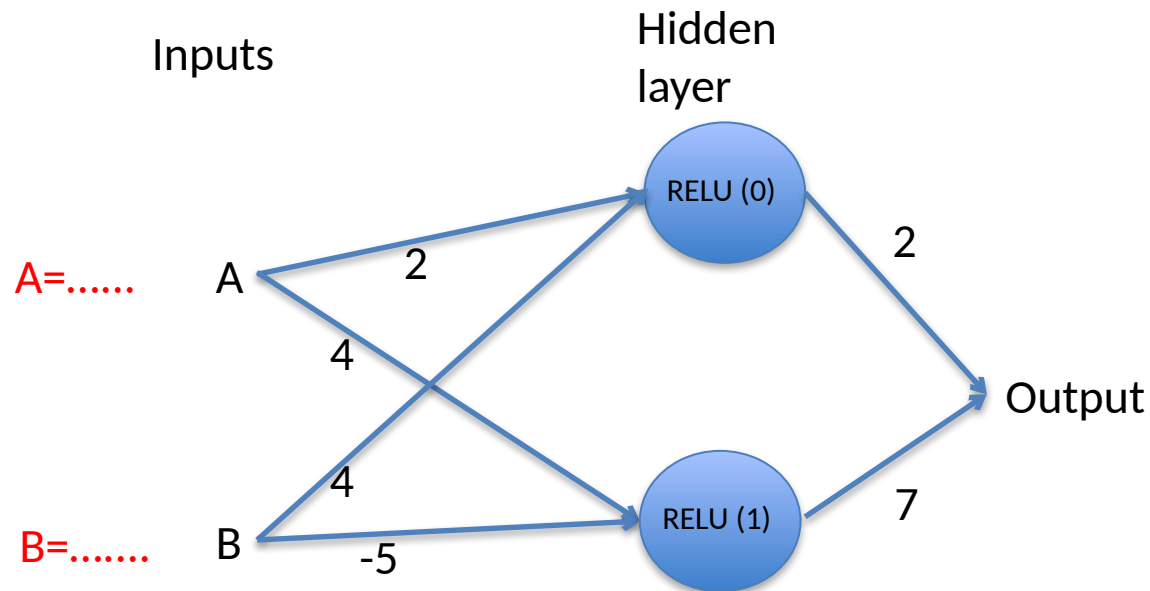
Utilicemos el mismo ejemplo anterior, pero ahora le aplicaremos una función de activación a cada neurona de la hidden layer:



Ver [2_activation_func_example.ipynb](#)

Activation Functions

Supongamos que tenemos varias instancias, no sólo una, y deseamos aplicar para cada una de esas instancias, el modelo NN para predecir el Output.



Ver 3_activation_func_muchas_example.ipynb

Lo mismo que antes , pero ahora vamos a definir una funcion que genera varias predicciones para varias instancias.

Redes profundas (deeper networks)

Redes profundas (deeper networks)

La única diferencia entre una NN (que ya hemos visto) y la famosa **DEEP NETWORK**, es que la segunda es simplemente un “fancy” name para referirse a una NN con MÁS DE 1 HIDDEN LAYER.

Redes profundas (deeper networks)

La única diferencia entre una NN (que ya hemos visto) y la famosa **DEEP NETWORK**, es que la segunda es simplemente un “fancy” name para referirse a una NN con MÁS DE 1 HIDDEN LAYER.

Sin embargo, el agregar más capas escondidas tiene una ventaja. Conforme agregamos más capas, la red tiene mayor capacidad de construir internamente representaciones complejas (no lineales) de patrones en los datos (de relaciones entre variables).

Redes profundas (deeper networks)

La única diferencia entre una NN (que ya hemos visto) y la famosa **DEEP NETWORK**, es que la segunda es simplemente un “fancy” name para referirse a una NN con MÁS DE 1 HIDDEN LAYER.

Sin embargo, el agregar más capas escondidas tiene una ventaja. Conforme agregamos más capas, la red tiene mayor capacidad de construir internamente representaciones complejas (no lineales) de patrones en los datos (de relaciones entre variables).

Si lo anterior es correcto, entonces la red podría eventualmente hacer muy buenas predicciones dado un set de atributos que prepresenten a una instancia.

Redes profundas (deeper networks)

La única diferencia entre una NN (que ya hemos visto) y la famosa DEEP NETWORK, es que la segunda es simplemente un “fancy” name para referirse a una NN con MÁS DE 1 HIDDEN LAYER.

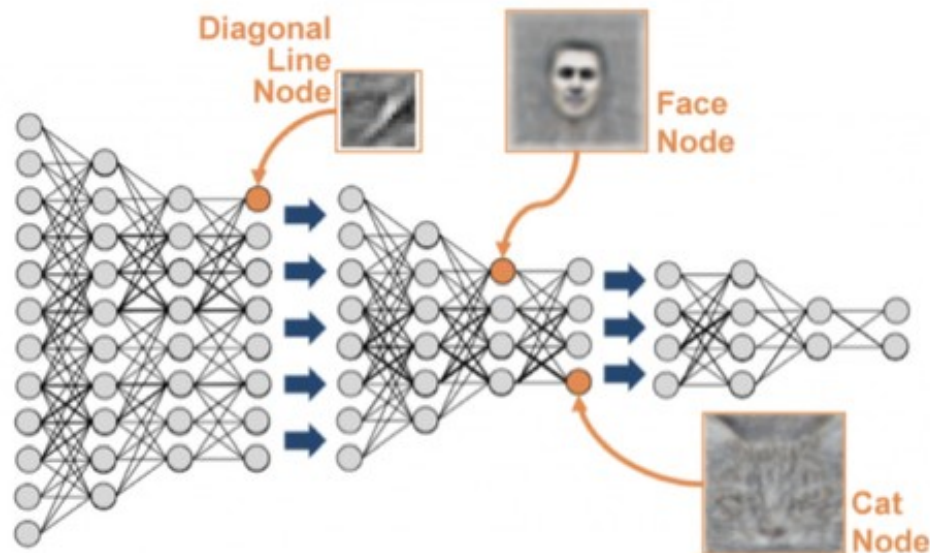
Sin embargo, el agregar más capas escondidas tiene una ventaja. Conforme agregamos más capas, la red tiene mayor capacidad de construir **internamente representaciones complejas (no lineales) de patrones en los datos (de relaciones entre variables).**

Si lo anterior es correcto, entonces la red podría eventualmente hacer muy buenas predicciones dado un set de atributos que prepresenten a una instancia.

Veremos algo más de esto de las representaciones en
1.4 complex_representations_nn.pdf

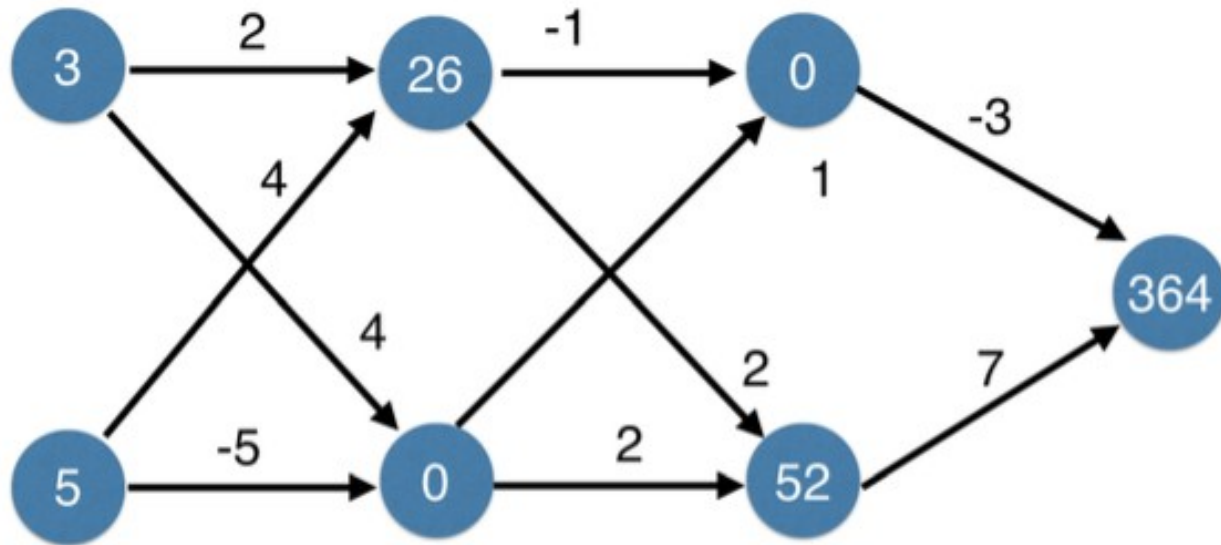
Redes profundas (deeper networks)

Algo interesante de estas **DEEP NETWORKS**, sin especificar en forma intencionada, la red “aprende” los pesos que existen entre cada conexión de neuronas, las cuales, como ya sabemos, especifican interacciones.



Redes profundas (deeper networks)

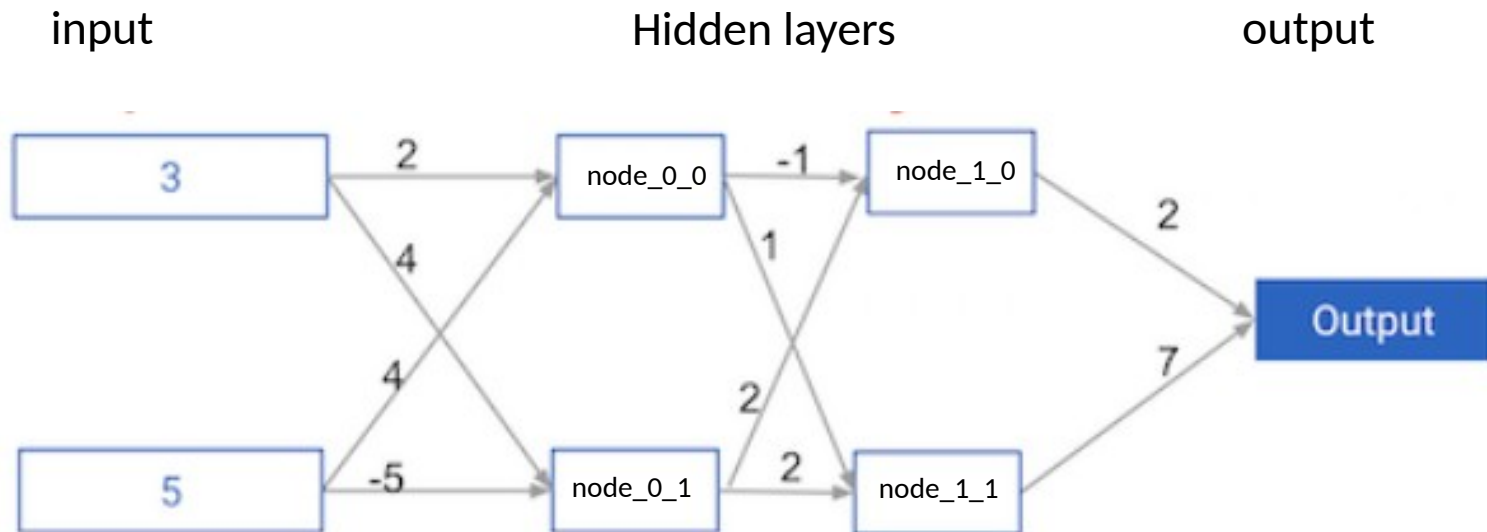
Sólo para asegurarnos de que está entendiendo la mecánica del forward propagation, verifique en esta NN con activaciones RELU que la predicción es correcta:



Entradas [3,5]

Redes profundas (deeper networks)

En `4_multilayer_example.ipynb` hacemos un código para practicar con una NN de dos capas.



Redes profundas (deeper networks)

MUY BIEN!!!!!!

Todo bien hasta aqui... pero.

¿cómo se encuentran los pesos de la red?

Redes profundas (deeper networks)

Antes de contestar esa pregunta, vamos a concentrarnos en la unidad fundamental de la NN, la neurona.

Veremos el PERCEPTRON.